

Exercício em duplas (10 min): qual método é chamado? (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

1 - Java

```
class Animal {
    String som() { return "..."; }
}
class Cachorro extends Animal {
    String som() { return "au"; }
}
Animal x = new Cachorro();
System.out.println(x.som());
```

2 - C++

```
struct A {
    void f() { cout << "A"; }
};
struct B : A {
    void f() { cout << "B"; }
};
B b; A *p = &b;
p->f();
```

SAÍDA:

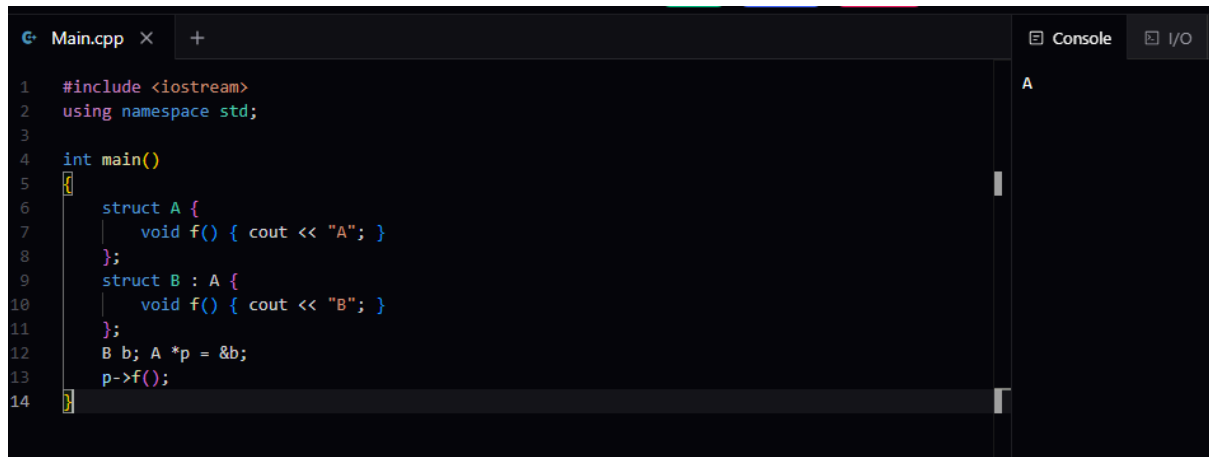


The screenshot shows a Java IDE with a file named Main.java. The code defines an Animal class with a som() method returning "...", and a Cachorro class that extends Animal and overrides som() to return "au". An instance of Cachorro is created and its som() method is called. The console output on the right shows "au".

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         class Animal {
6             String som() { return "..."; }
7         }
8         class Cachorro extends Animal {
9             String som() { return "au"; }
10        }
11        Animal x = new Cachorro();
12        System.out.println(x.som());
13    }
14 }
```

Console: au

Vinculação dinâmica. A mensagem escolhe o método.



The screenshot shows a C++ IDE with a file named Main.cpp. The code defines a struct A with a f() method that prints "A", and a struct B that inherits from A and overrides f() to print "B". An instance of B is created, and a pointer of type A is assigned to it. The f() method is called through the pointer. The console output on the right shows "A".

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     struct A {
7         void f() { cout << "A"; }
8     };
9     struct B : A {
10         void f() { cout << "B"; }
11     };
12     B b; A *p = &b;
13     p->f();
14 }
```

Console: A

Vinculação estática. "A" não é virtual.

Exercício: qual método é chamado? (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

3 - Java

```
class A { String nome = "A";  
    String getNome() { return nome; } }  
class B extends A { String nome = "B";  
    String getNome() { return nome; } }  
A x = new B();  
System.out.println(x.nome + " " + x.getNome());
```

4 - Python

```
class Contador:  
    total = 0  
    def __init__(self):  
        Contador.total += 1  
        self.id = Contador.total  
  
a = Contador(); b = Contador()  
print(a.id, b.id, a.total)
```

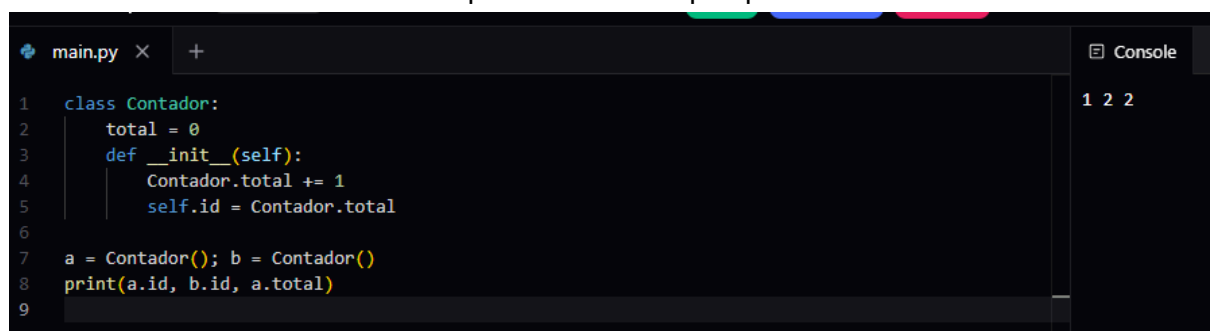
SAÍDAS:



```
J Main.java x +  
1 import java.util.*;  
2  
3 public class Main {  
4     public static void main(String[] args) {  
5         class A { String nome = "A";  
6             String getNome() { return nome; } }  
7         class B extends A { String nome = "B";  
8             String getNome() { return nome; } }  
9         A x = new B();  
10        System.out.println(x.nome + " " + x.getNome());  
11    }  
12 }
```

Console
A B

CAMPOS não são polimórficos: `x.nome` usa o tipo declarado (A), e o campo `nome` de B apenas **ESCONDE** o de A. Já o **MÉTODO** `getNome()` é despachado dinamicamente e devolve o nome de B. É um motivo para manter campos privados.



```
main.py x +  
1 class Contador:  
2     total = 0  
3     def __init__(self):  
4         Contador.total += 1  
5         self.id = Contador.total  
6  
7 a = Contador(); b = Contador()  
8 print(a.id, b.id, a.total)  
9
```

Console
1 2 2

`Total` é uma variável de **CLASSE** (uma cópia só, compartilhada), e `id` é de **INSTÂNCIA** (uma por objeto). `a.total` encontra o total da classe.

Exercício: qual método é chamado? (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

5 - Java

```
class A {
    static String quem() { return "A"; }
}
class B extends A {
    static String quem() { return "B"; }
}
A x = new B();
System.out.println(x.quem());
```

6 - Go

```
type Animal struct{}
func (Animal) Som() string { return "..."}
func (a Animal) Falar() string {
    return "faz " + a.Som()
}
type Cao struct{ Animal }
func (Cao) Som() string { return "au" }

fmt.Println(Cao{}.Falar(), Cao{}.Som())
```

SAÍDAS:



The screenshot shows an IDE with a file named 'Main.java'. The code defines a base class 'A' with a static method 'quem()' returning 'A', and a subclass 'B' that extends 'A' and overrides 'quem()' to return 'B'. In the 'main' method, an instance of 'B' is created and assigned to a variable of type 'A'. The output in the console is 'A'.

Métodos static não são sobrescritos, e sim ESCONDIDOS. A chamada é resolvida na compilação pelo tipo declarado (A). O próprio compilador (ou a IDE) avisa para chamar como A.quem().



The screenshot shows an IDE with a file named 'main.go'. The code defines an 'Animal' struct with a 'Som()' method returning '...', and a 'Cao' struct that embeds 'Animal' and overrides 'Som()' to return 'au'. In the 'main' function, a 'Cao' instance is created and its 'Falar()' method is called, which internally calls 'Som()' on the embedded 'Animal' struct. The output in the console is 'faz ... au'.

Embedding NÃO é herança: dentro de Animal.Falar, a.Som() chama o Som de Animal, porque não há despacho dinâmico entre o struct embutido e o que o contém. Cao{}.Som() chama o Som de Cao. Para ter polimorfismo em Go, é preciso usar uma interface.

Fechamento: das 6 questões, quais foram decididas na COMPILAÇÃO? A 2 (C++ sem virtual), o campo da 3, a 5 (static) e a 6 (Go sem interface). Só a 1 e o getNome() da 3 usam despacho dinâmico.

